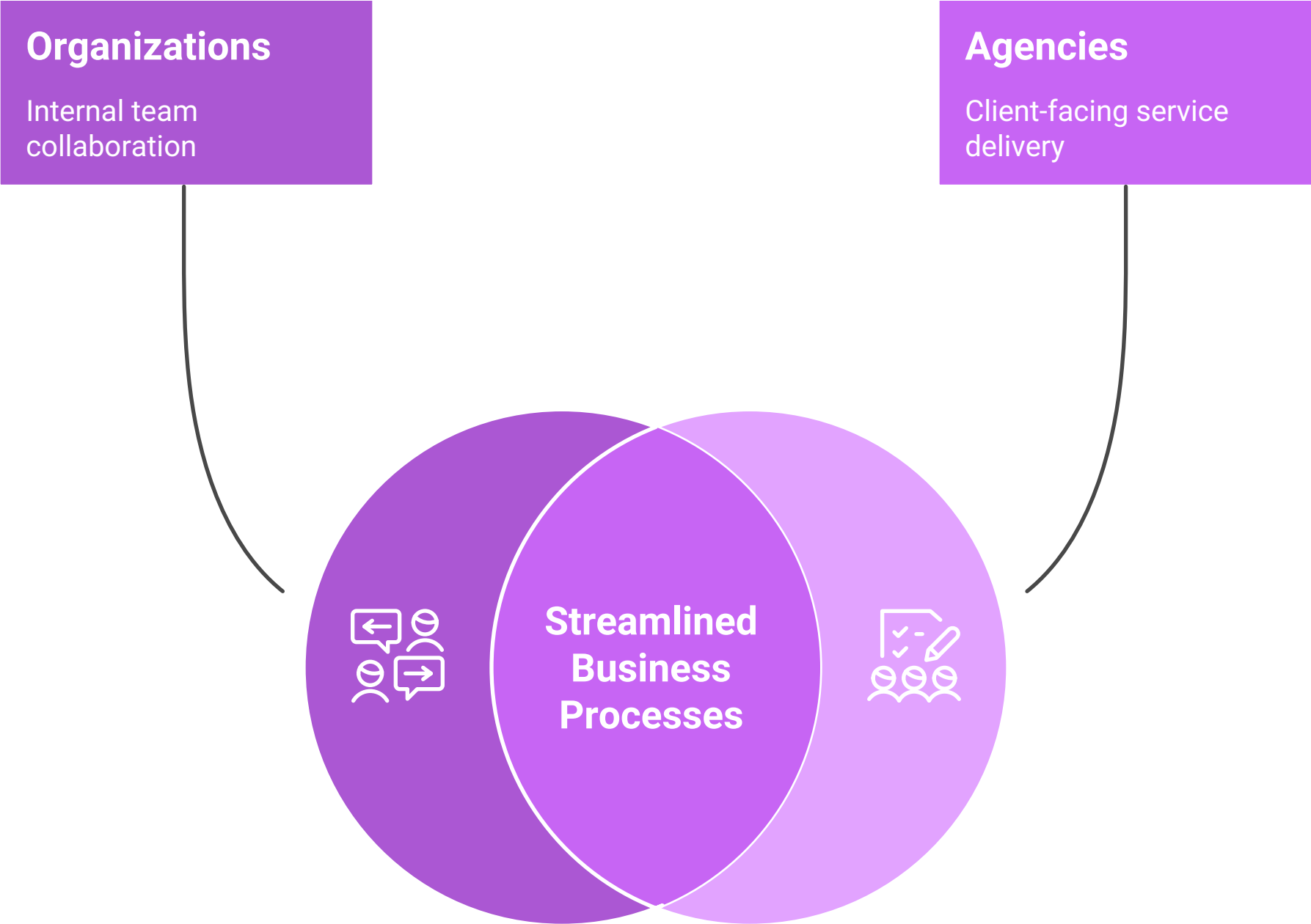




# Multi-Tenant AI Agent Platform

- **Subtitle:** Scalable, Automated 1-to-1 Communication for Organizations, Agencies, and Clients.
- **Theme:** Modern, Enterprise AI, Connectivity.

## Unified AI Communication for All



Made with  Napkin

## The Problem Statement

- **Communication Bottleneck:** Large organizations manage hundreds of agencies; agencies manage thousands of sellers/clients.
- **The Scalability Gap:** Manual 1-to-1 support is slow, expensive, and prone to human error.
- **Information Silos:** Data is scattered across different platforms, making it hard for agencies to give quick, accurate answers.

## Overcoming Communication and Scalability Challenges

- **Communication Bottleneck**  
Hundreds of agencies, thousands of sellers
- **Scalability Gap**  
Manual support is slow and expensive
- **Information Silos**  
Data scattered across platforms



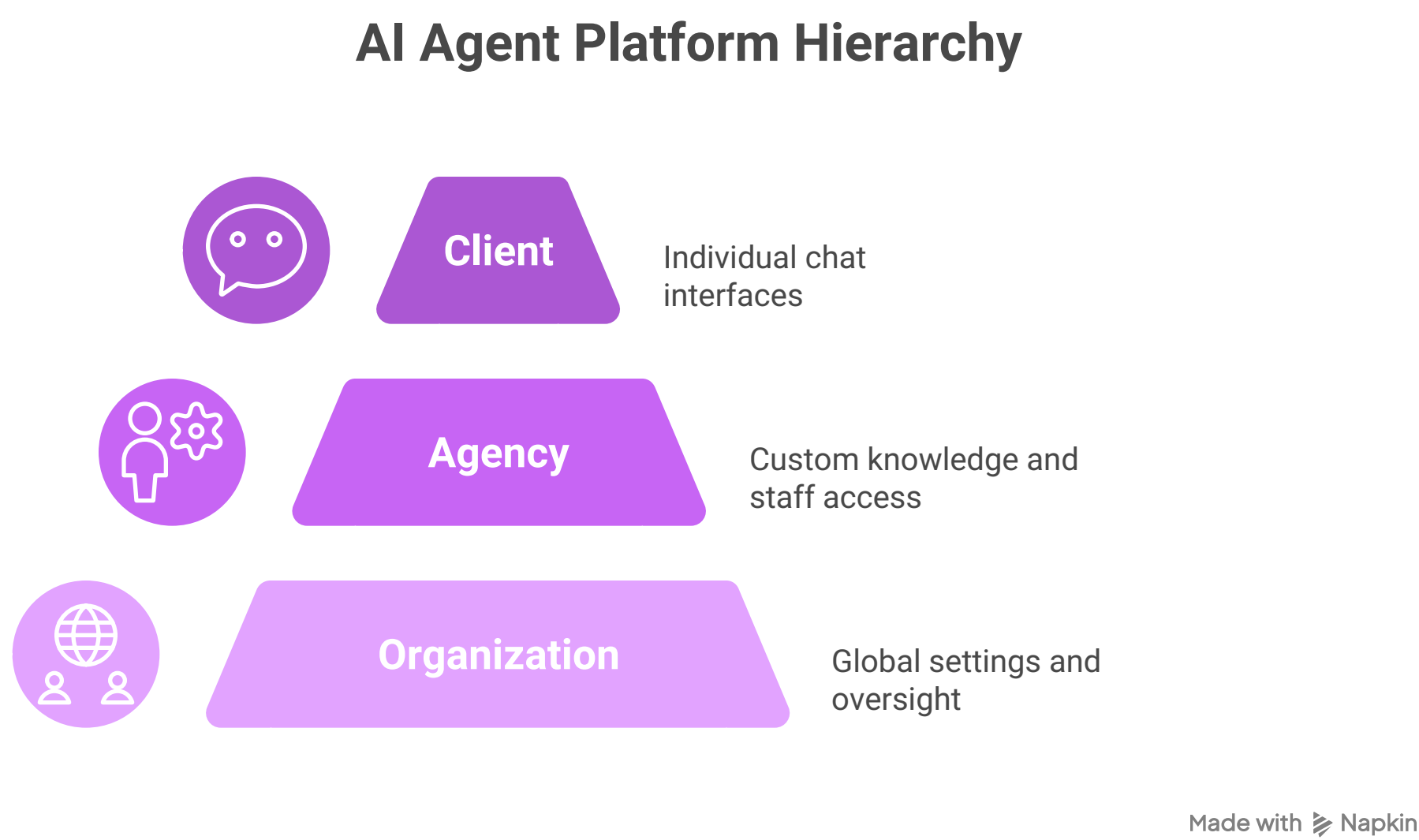
Made with  Napkin

Slide 3: The Solution – "The AI-Hub"

- **Centralized Control:** One platform for the Parent Organization.
- **Agency Autonomy:** Each agency gets a private "sub-account" with its own AI.
- **AI-Powered Efficiency:** Sellers get 24/7 instant responses tailored to their specific agency’s rules and data.

Slide 4: High-Level Architecture

- **Level 1: The Root [Organization]:** Global settings, billing, and oversight.
- **Level 2: The Tenant [Agency]:** Custom knowledge bases, agency-specific prompt engineering, and staff access.
- **Level 3: The Client [Seller]:** Individual chat interfaces [WhatsApp, Web, API].



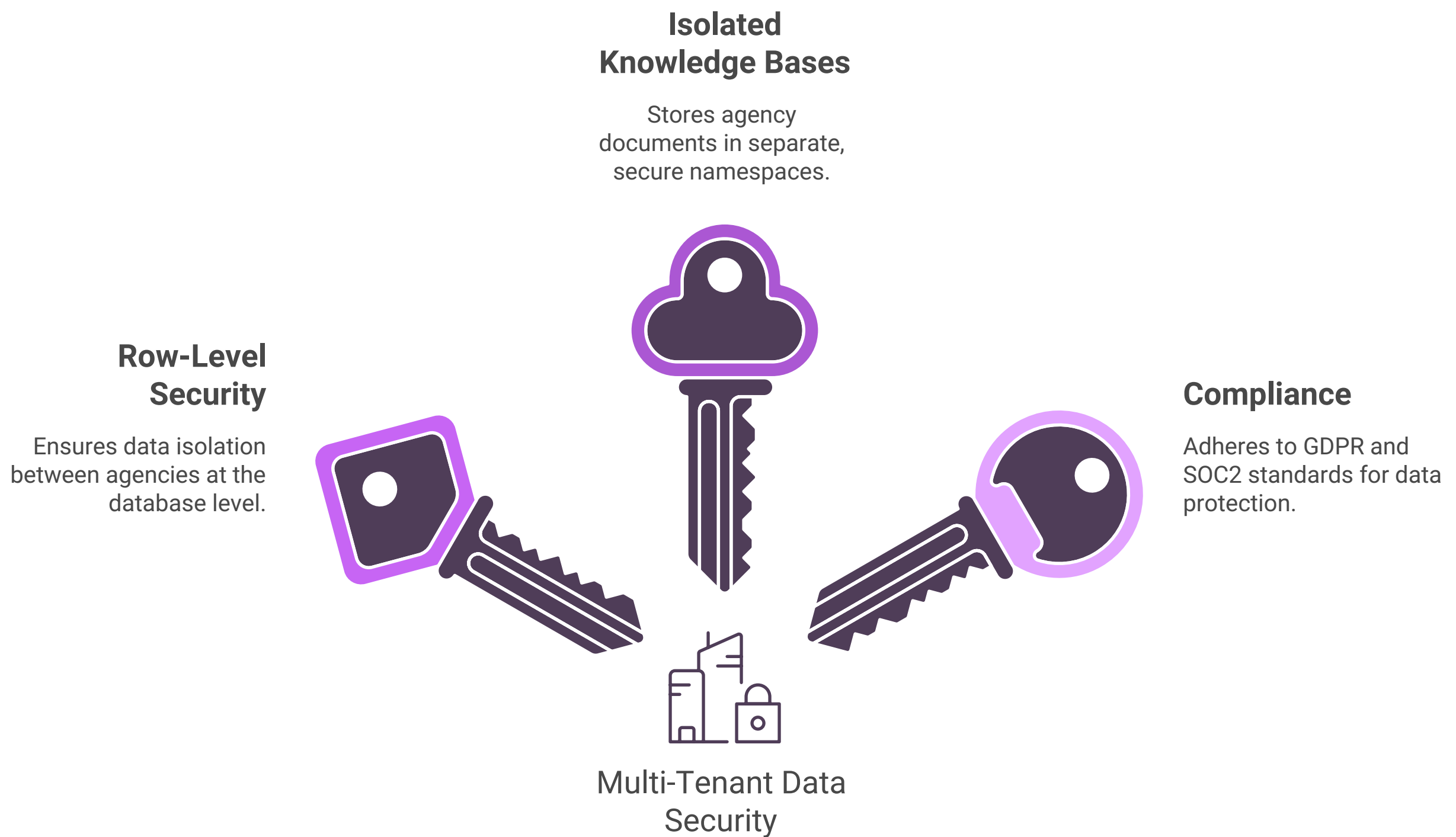
Slide 5: Technical Flow (The "Brain")

1. **Inbound:** Seller sends a query via WhatsApp/Web.
2. **Context Recognition:** System identifies the Agency\_ID and Client\_ID.
3. **RAG Layer [Retrieval Augmented Generation]:** System pulls specific SOPs/Data for that Agency.
4. **AI Synthesis:** LLM generates a response using the Agency’s "Tone of Voice."
5. **Outbound:** Response delivered in < 2 seconds.

Slide 6: Multi-Tenant Data Security

- **Row-Level Security [RLS]:** Database ensures Agency A can never access Agency B’s logs.
- **Isolated Knowledge Bases:** Each agency’s PDFs/Docs are stored in separate Vector namespaces.
- **Compliance:** GDPR/SOC2 ready framework.

# Multi-Tenant Data Security



Made with  Napkin

## Slide 7: Automation & Human-in-the-Loop

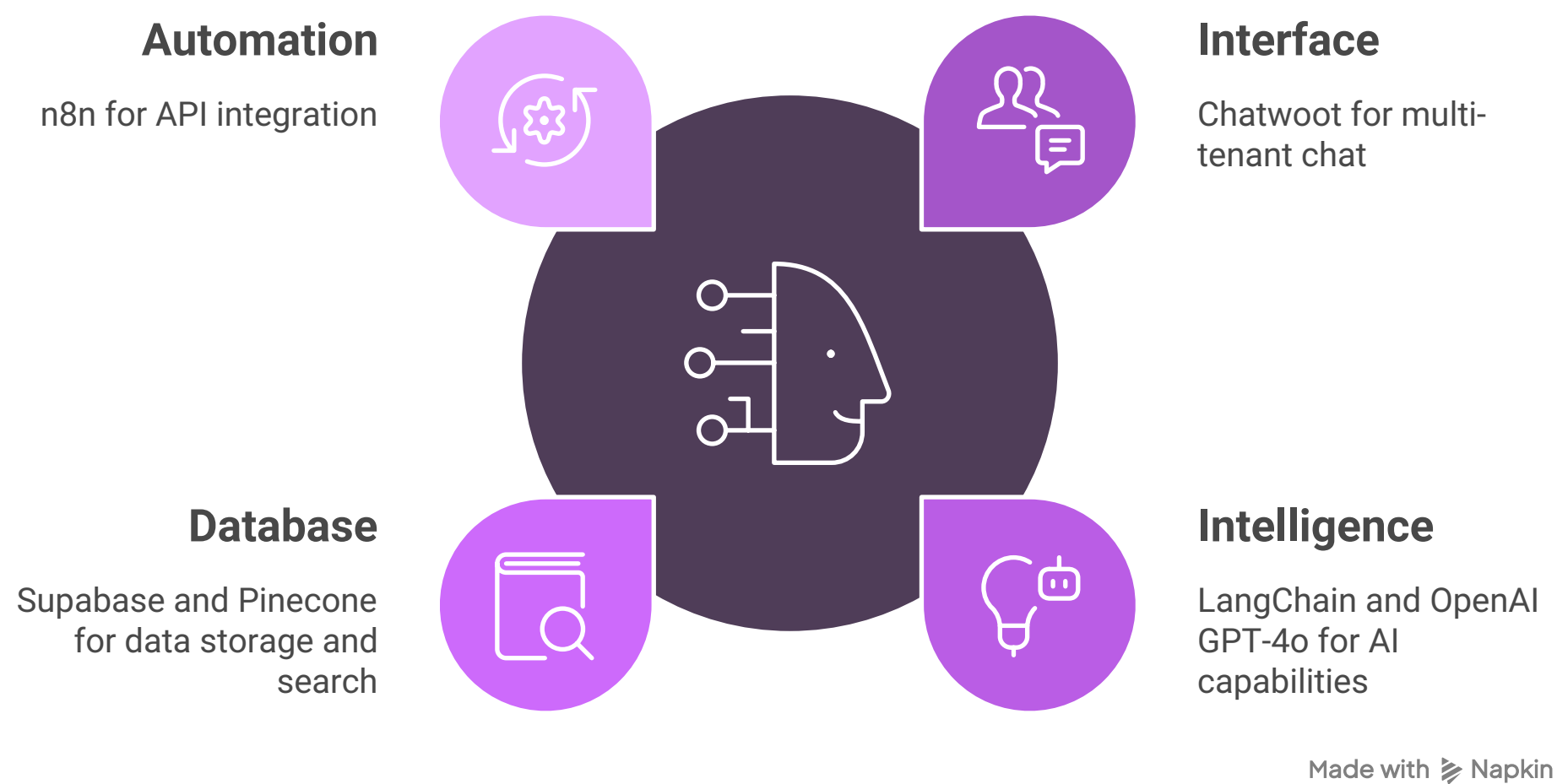
- **AI-First Policy:** AI handles 80% of routine queries.
- **Smart Escalation:** If AI confidence is low (<70%), the chat is instantly flagged for a human agent at the Agency.
- **Sentiment Analysis:** If the seller is angry, the AI alerts the Agency manager immediately.

## Slide 8: The Tech Stack (Build-Fast Strategy)

- **Interface:** Chatwoot [Open-source multi-tenant chat].
- **Intelligence:** LangChain / OpenAI GPT-4o.
- **Database:** Supabase [PostgreSQL] + Pinecone [Vector Search].
- **Automation:** n8n [To glue Org-specific APIs together].



# Components of the AI Agent Platform



## Slide 9: Scalability & Use Cases

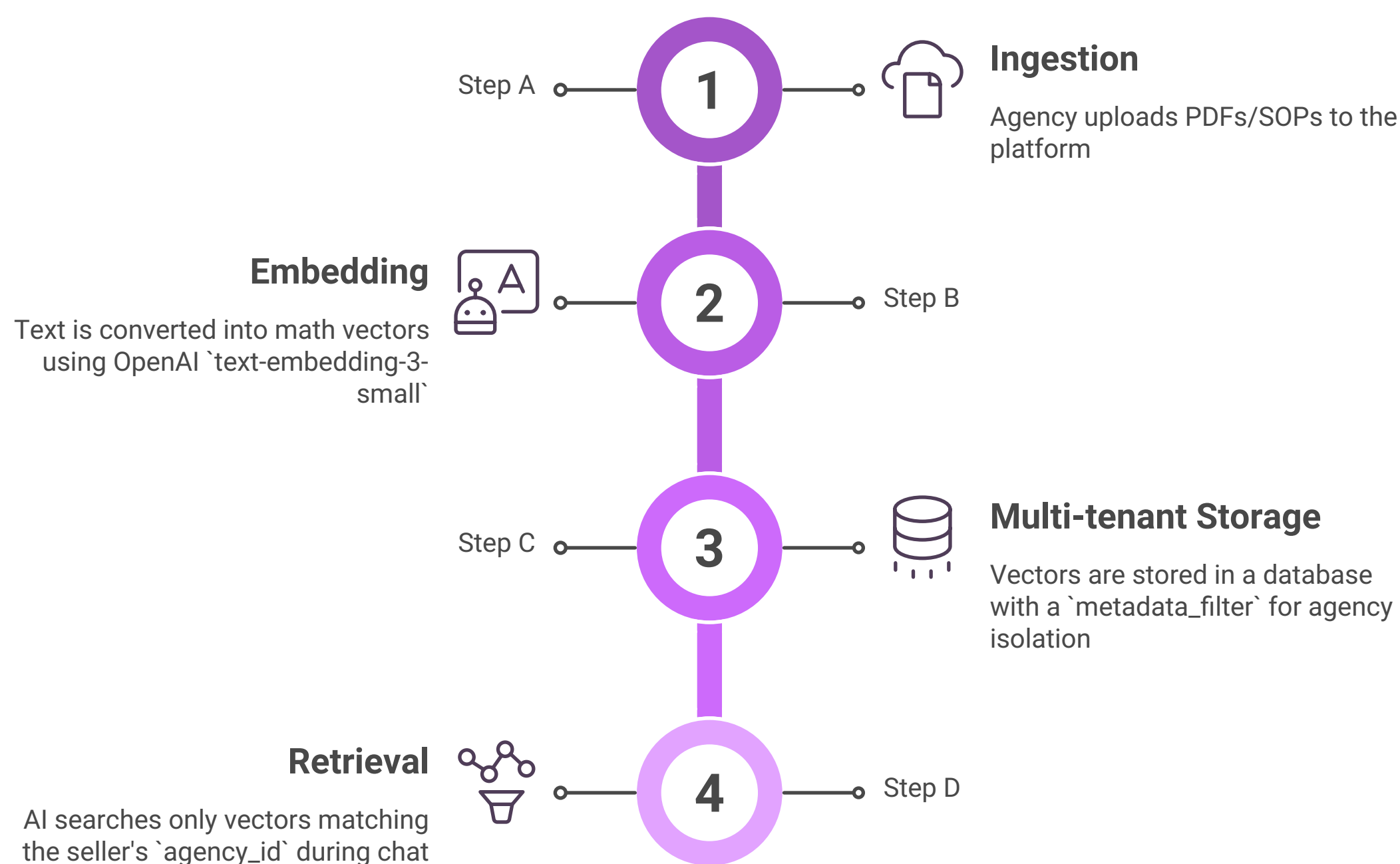
- **Beyond Amazon:** Applicable to Franchise models, Real Estate Networks, Logistics Hubs, and Global Retailers.
- **White-Labeling:** Agencies can put their own logo on the chat tool they give to sellers.

## 10 PPT Content: Advanced Flow & Tech

**Slide 11: The "Intelligence" Flow (RAG Architecture)** Explain how the AI doesn't "leak" data between agencies.

- **Step A: Ingestion:** Agency uploads PDFs/SOPs.
- **Step B: Embedding:** Text is converted into math vectors (using OpenAI text-embedding-3-small).
- **Step C: Multi-tenant Storage:** Vectors are stored in a database with a metadata\_filter [e.g., {"agency\_id": "agency\_001"}].
- **Step D: Retrieval:** When a seller chats, the AI *only* searches vectors matching that agency\_id.

# Intelligence Flow (RAG Architecture)



Made with Napkin

**Slide 12: The Tech Stack (Scratch vs. Fast-Track)** | Component | Enterprise (Scratch) | Rapid Prototyping (Fast) | | :--- | :--- | :--- | | **Language** | Python (FastAPI) | Node.js (Next.js) | | **AI Orchestration** | LangChain / LangGraph | **Dify.ai** (Visual workflow) | | **Vector DB** | Milvus / Weaviate | **Pinecone** (Serverless) | | **Database** | PostgreSQL (RDS) | **Supabase** (Postgres + Auth) | | **Messaging** | Custom WebSockets | **Chatwoot** (Omnichannel API) |

## 2. Free & Open Source Tools (The "Cheat Sheet")

If you need to show "How we will do it fast," list these:

1. **Dify.ai**: (Self-hosted) Best open-source platform to build AI apps. It handles multi-tenancy, RAG, and Agent flows out of the box.
2. **Chatwoot**: (Self-hosted) Gives you the entire dashboard and mobile app for agencies.
3. **Onyx (formerly Danswer)**: The best open-source enterprise search/RAG tool. Connects to Slack, Google Drive, etc., instantly.
4. **n8n.io**: Use the self-hosted version to automate the flow between Amazon APIs and your AI.

## 3. Database Schema (For the Technical Slide)

Put this in a small box or diagram. It proves you've thought about the logic.  
SQL

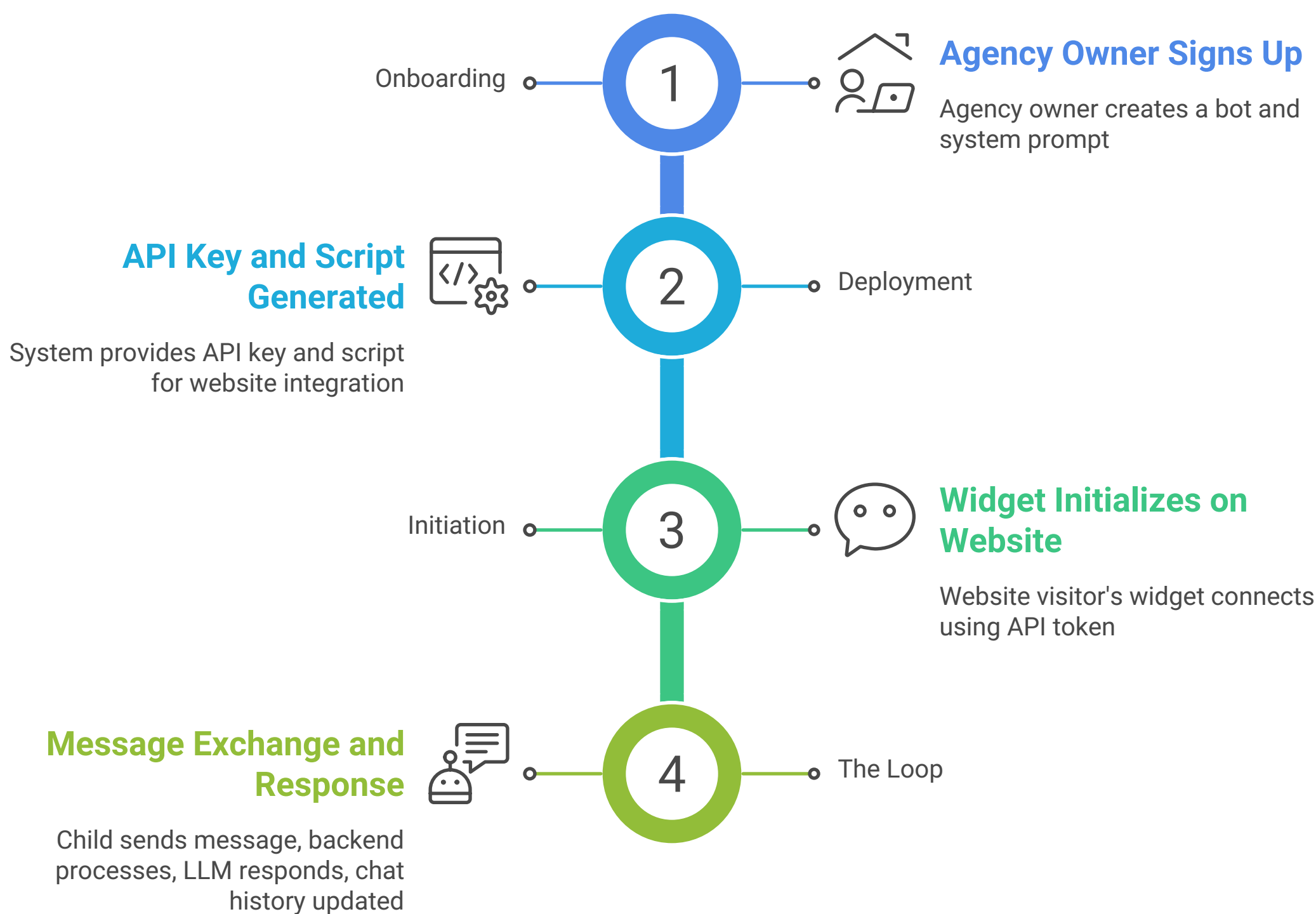
```
-- The 3-Tier Hierarchy
Table Organizations { id, name, api_key }
Table Agencies { id, org_id, agency_name, custom_prompt }
Table Sellers { id, agency_id, seller_name, platform_type }
Table Conversations { id, seller_id, agency_id, last_msg, status }
Table KnowledgeBase { id, agency_id, file_url, vector_id }
```

- **Unique Selling Point (USP)**: "This isn't a chatbot; it's a **Managed AI Service Layer** that empowers agencies to scale 10x without adding headcount."

## How Your Project Will Work (The Lifecycle)

1. **Onboarding:** An **Agency Owner** signs up and creates a "Bot." They give it a name and a "System Prompt" [e.g., "You are a helpful assistant for a Real Estate agency"].
2. **Deployment:** Your system generates a unique **API Key/Token** and a <script> tag. The Agency copies this into their website's HTML.
3. **Initiation:** When a visitor (the **Child**) loads the website, the widget initializes using the token to identify which Agency it belongs to.
4. **The Loop:**
  - The Child types a message.
  - The Widget sends that message + Agency Token to your **Backend API**.
  - The Backend fetches the specific **System Prompt** for that token from the **Data Store**.
  - The Backend sends the prompt and message to the **LLM Layer**.
  - The LLM generates a response, which the Backend saves to the **Chat History** and sends back to the Widget.

## Multi-Tenant AI Agent Platform Lifecycle



Made with Napkin

### 1. The "Widget" (Client-side Injection)

Tumhe ek bundled JS file banani hai jo Agency ki site par load hogi.

- **Logic:** Ye file ek <iframe> inject karegi [taaki agency ki CSS tumhare chat UI ko kharab na kare].
- **Auth:** Script tag se data-agent-id uthao.
- **Communication:** Iframe ke andar se tumhare backend par **WebSockets** ([Socket.io](#)) ya **Server-Sent Events (SSE)** ka use karo real-time streaming ke liye.

### 2. Backend API (The Orchestrator)



1 aur tumhara asli logic hainega.

- **Tenant Middleware:** Har request ka Agent-ID check karo. Database se us agent ka specific system\_prompt aur temperature fetch karo.
- **Session Management:** Visitor ke liye ek session\_id generate karo (Redis mein phenk do 30 mins expiry ke sath) taaki chat context maintain rahe.
- **The LLM Call (Node/Python):**
- JavaScript

```
// Pseudocode for the core logic
const chatHistory = await db.messages.find({ sessionId });
const response = await openai.chat.completions.create({
  model: "gpt-4-turbo",
  messages: [
    { role: "system", content: agent.systemPrompt }, // Agency specific
    ...chatHistory,
    { role: "user", content: userInput }
  ],
  stream: true, // User ko "typing" feel dene ke liye
});
```

### 3. Data Schema (PostgreSQL Recommended)

Tumhe relational data chahiye kyunki multi-tenancy mein mapping zaroori hai.

- Table: Agencies: id, api\_key, webhook\_url, created\_at
- Table: Bots: id, agency\_id, system\_prompt, model\_name, theme\_config (JSON)
- Table: ChatSessions: id, bot\_id, visitor\_id, metadata (IP, Browser)
- Table: Messages: id, session\_id, role (user/assistant), content, timestamp

### 4. Step-by-Step Implementation Workflow

1. **Step 1 (API Design):** Ek POST endpoint banao /chat jo agent\_id, session\_id, aur message le. Pehle static response bhejo, phir OpenAI SDK integrate karo.
2. **Step 2 (Context Isolation):** Backend mein logic likho ki Messages table se sirf wahi last 5-10 messages uthaye jo current session\_id ke hain. Isse context leak nahi hoga.
3. **Step 3 (The Script Tag):** Ek widget.js likho jo DOM manipulate karke ek chat window banaye. Isko CDN ya apne server se serve karo.
4. **Step 4 (Admin Dashboard):** Ek simple React/Next.js app banao jaha Agency owner UPDATE bots SET system\_prompt = '...' WHERE agency\_id = 123 kar sake.

### 5. Deployment Hardware

- **Backend:** Dockerize your app. Deploy on AWS [EC2] or DigitalOcean.
- **Database:** Managed Postgres (Supabase ka use kar lo, kaam asaan ho jayega).
- **Cache:** Redis for session state management.

